

RichTerm - Bedienungsanleitung

Chat mit Claude Code (oder einer lokalen KI) mit gerenderten Formeln, Diagrammen und Animationen, plus Terminal, in einem Fenster. Stand: Oktober 2026.

- | | |
|---|--|
| 1. Was RichTerm ist | 7. Berechtigungen |
| 2. Starten | 8. Das Terminal und der Befehl <code>rt</code> |
| 3. Die Oberfläche | 9. Tastenkürzel |
| 4. Das Profil: <code>richterm.md</code> | 10. Typische Arbeitsabläufe |
| 5. Modelle wählen und laden | 11. Probleme und Lösungen |
| 6. Gedächtnis und Sitzungen | 12. Dateien im Überblick |

1. Was RichTerm ist

RichTerm ist ein Fenster mit zwei Arten von Tabs:

- **Chat:** Du schreibst unten, die KI antwortet oben. Die Antworten werden gerendert wie in einem Browser-Chat: Markdown, LaTeX-Formeln in Buchqualität, Tabellen, Mermaid-Diagramme, Bilder und HTML-Blöcke als Live-Vorschau (Animationen, interaktive Grafiken).
- **Terminal:** ein vollwertiges Terminal (dieselbe Engine wie GNOME Terminal), beliebig viele Tabs.

Die KI arbeitet in einem **Arbeitsordner**. Mit Claude Code als Backend kann sie die Dateien darin lesen, Code ausführen und Dateien schreiben, etwa Vorlesungsfolien lesen und dazu Fragen beantworten. In jedem Ordner liegen zwei Dateien, die RichTerm selbst anlegt:

`richterm.md` (das Profil: Rolle, Verhalten, Modell) und `richterm-verlauf.md` (das Gedächtnis).

Es ist nichts installiert worden: RichTerm besteht aus Python und den Bibliotheken, die Linux Mint ohnehin mitbringt. Das Projekt liegt unter `~/Desktop/RichTerm` und ist ein Git-Repository.

2. Starten

Im aktuellen Ordner (der übliche Weg)

```
cd ~/Uni/Maschinelles_Lernen    # Ordner mit Folien, Übungen, Notizen
richterm                        # RichTerm öffnet sich mit diesem Ordner
```

Das Terminal ist sofort wieder frei und darf geschlossen werden. Mit Pfad geht es von überall: `richterm ~/Uni/ML`. Jeder Aufruf öffnet ein eigenes Fenster; du kannst also ein RichTerm für die Vorlesung und eins für ein Projekt gleichzeitig offen haben.

Weitere Wege

- **Dateimanager (Nemo):** Rechtsklick auf einen Ordner oder in einen Ordner hinein, „**RichTerm hier öffnen**“.
- **Desktop-Icon / Anwendungsmenü:** startet im Home-Ordner; den Ordner wechselt du danach über den Ordnerknopf oben links.

Beim ersten Start in einem Ordner werden `richterm.md` und `richterm-verlauf.md` angelegt. Die Statuszeile unter dem Eingabefeld weist darauf hin. Fülle das Profil aus (Abschnitt 4), bevor du ernsthaft arbeitest.

3. Die Oberfläche

Element	Funktion
Ordnerknopf (oben links)	Zeigt den Arbeitsordner; Klick öffnet die Ordnerwahl. Ein Wechsel startet eine neue Sitzung.
Modell	Standard / Opus / Sonnet / Haiku. Gesperrt, wenn das Profil ein Modell festlegt.
Berechtigungen	Fragen / Dateien bearbeiten erlauben / Alles erlauben (Abschnitt 7). Gesperrt, wenn das Profil es festlegt.
Profil	Öffnet <code>richterm.md</code> im Editor.
Neu laden	Liest das Profil neu ein (nach dem Bearbeiten). Gilt ab der nächsten Nachricht.
* / ☾	Hell / dunkel.
Neuer Chat	Beendet die laufende Sitzung und beginnt frisch (Abschnitt 6).
Eingabefeld	<code>Enter</code> sendet, <code>Shift+Enter</code> macht einen Zeilenumbruch. <code>■</code> bricht eine laufende Antwort ab.
Statuszeile	Über dem Eingabefeld: Ordner, Sitzung, Dauer und Kosten der letzten Antwort, Hinweise.

In den Antworten

- **Graue Zeilen** sind Werkzeugaufrufe (Datei gelesen, Befehl ausgeführt). Ein Klick zeigt Details und das Ergebnis.
- **Live-Vorschau** bei HTML-Blöcken: *Code* zeigt den Quelltext, *Neu laden* startet die Animation neu, *Im Browser* öffnet die Seite groß.
- **Berechtigungsfrage:** ein Kasten mit *Erlauben* / *Ablehnen*, wenn die KI ein Werkzeug benutzen will, das nicht freigegeben ist.
- `Ctrl` + `+` / `-` ändert die Schriftgröße des Chats.

4. Das Profil: richterm.md

Das Profil ist die wichtigste Datei. Es wird bei jedem Start gelesen und der KI als **verbindliche Anweisung** mitgegeben, egal ob Claude oder eine lokale KI antwortet. Es gilt nur für diesen Ordner. So verhält sich die KI im Vorlesungsordner wie ein Tutor und im Projektordner wie ein Entwickler.

Oben: Einstellungen

Feld	Werte	Bedeutung
backend	claude · command	claude = Claude Code, voller Agent mit Dateizugriff. command = beliebige andere KI mit Kommandozeile, nur Chat.
model	opus · sonnet · haiku · leer; bei command der Modellname, z. B. qwen2.5:7b	Leer = Auswahl in der Kopfzeile gilt.
command	z. B. ollama run {model}	Nur für command . {model} wird ersetzt.
permissions	default · acceptEdits · bypassPermissions	default = Kopfzeile entscheidet; sonst festgelegt und gesperrt.
allowed_tools	z. B. Read Grep Glob	Nur diese Werkzeuge: hier reines Lesen, nichts wird geändert.
disallowed_tools	z. B. WebSearch "Bash(rm *)"	Diese Werkzeuge verbieten.
language	z. B. Deutsch	Sprache der Antworten.
history	true · false	Gedächtnis (Verlaufsdatei) an oder aus.
context_chars	Zahl, Standard 20000	Wie viel Verlauf beim Start mitgegeben wird. Claude: 20000-60000; kleine lokale Modelle: 4000-8000.

Darunter: Anweisungen in normaler Sprache

Die Vorlage enthält sieben Abschnitte mit Beispielen. Schreib sie einfach um, lösche, was nicht passt:

- **Rolle:** Tutor, Entwickler, Reviewer, Prüfungsvorbereiter, Schreibassistent ...
- **Themengebiet und Ziel:** „Vorlesung ML, Klausur im Februar, Schwerpunkt neuronale Netze.“
- **Was in diesem Ordner liegt:** „folien/ = PDFs, uebungen/ = Übungsblätter, notizen.md = meine Notizen.“
- **Verhalten:** schrittweise erklären, Rückfragen stellen, Quellen aus dem Ordner nennen.

- **Genauigkeit und Kreativität:** keine erfundenen Fakten; „steht in den Unterlagen“ vs. „Einschätzung“ trennen.
- **Antwortformat:** LaTeX-Formeln, nummerierte Herleitungen, Diagramme, Länge.
- **Nicht erlaubt:** nichts löschen, Folien nur lesen ...

Nach jeder Änderung: im Chat auf *Neu laden* klicken. Die Änderung gilt ab der nächsten Nachricht (es wird eine neue Sitzung gestartet; der Verlauf bleibt erhalten).

5. Modelle wählen und laden

Claude-Modelle

Mit `backend: claude` stehen **Opus** (stärkstes, teuerstes), **Sonnet** (ausgewogen) und **Haiku** (schnell, günstig) zur Wahl, entweder in der Kopfzeile oder fest im Profil (`model: sonnet`). Für Lernen mit Herleitungen empfiehlt sich Sonnet oder Opus, für schnelle Nachfragen Haiku. Die Kosten der letzten Antwort stehen in der Statuszeile. Ein Modellwechsel startet eine neue Sitzung; der Kontext kommt aus der Verlaufsdatei (Abschnitt 6).

Lokale Modelle (Ollama)

RichTerm bringt ein eigenes Ollama mit, ohne Installation, unter `~/.local/share/ollama` . Es läuft auf Port 11435 mit Modellen unter `~/.local/share/ollama/models` und wird bei Bedarf automatisch gestartet. Ein eventuell vorhandener älterer System-Ollama-Dienst (Port 11434) wird nicht benutzt.

Ein neues Modell laden: ein RichTerm-Terminal öffnen (`Ctrl+Shift+T`), dort stimmen Pfad und Port automatisch) und eingeben:

```
ollama pull qwen2.5:7b      # lädt das Modell (ca. 4,7 GB), einmalig
ollama list                # zeigt die geladenen Modelle
ollama rm qwen2.5:0.5b     # löscht ein Modell wieder
```

Modellnamen findest du auf `ollama.com/library` . Dann im Profil:

```
backend: command
model: qwen2.5:7b
command: ollama run {model}
context_chars: 6000
```

Neu laden, fertig. Die Antworten werden genauso gerendert wie bei Claude.

Modell	Größe	Eignung auf deinem Laptop (CPU, 31 GB RAM)
qwen2.5:0.5b	0,4 GB	Nur zum Testen; antwortet in Sekunden, ignoriert Kontext oft.
qwen2.5:3b	1,9 GB	Schnell, einfaches Niveau.
qwen2.5:7b	4,7 GB	Beste Mischung aus Deutsch, Mathe und Code; 4-8 Wörter/s.
llama3.1:8b	4,9 GB	Solide Allzweck-Wahl.
qwen2.5:14b	9 GB	Besser, aber nur 1-3 Wörter/s.

Grenzen lokaler Modelle: keine Dateiwerkzeuge (sie können den Ordner nicht selbst lesen), deutlich schwächer als Claude bei längeren Herleitungen, aber kostenlos und vollständig privat. Nur das Claude-Backend liest Folien, führt Code aus und schreibt Dateien.

Andere KI mit Kommandozeile

backend: `command` funktioniert mit jedem Programm, das den Text auf stdin liest und die Antwort auf stdout ausgibt. RichTerm übergibt Profil und bisherigen Gesprächsverlauf als Text, z. B. `command: llm -m {model}`.

6. Gedächtnis und Sitzungen

Es gibt zwei Ebenen, die zusammenarbeiten:

Claude-Code-Sitzung

Claude Code speichert jedes Gespräch selbst (unter `~/.claude/projects/...`), mit allem, was es gelesen und getan hat. RichTerm merkt sich pro Ordner die letzte Sitzung und **setzt sie beim nächsten Start automatisch fort** (Statuszeile: „Letzte Claude-Sitzung wird fortgesetzt“). *Neuer Chat* beendet das und beginnt frisch.

Die Verlaufsdatei richterm-verlauf.md

Das modellunabhängige Gedächtnis im Ordner. Jede Frage und Antwort wird angehängt, mit Zeit, Modell und LaTeX-Formeln. Beim Start einer neuen Sitzung, also nach einem Modell- oder Backend-Wechsel oder nach *Neuer Chat*, bekommt die KI die **Zusammenfassung** oben in der Datei und den **jüngsten Verlauf** mit. Nur darüber weiß Sonnet, was du mit Haiku besprochen hast, oder ein lokales Modell, was Claude erklärt hat. Beim Start zeigt der Chat die letzten Einträge grau an.

Automatische Verdichtung: Wird der Rohverlauf länger als das Budget (`context_chars`), fasst das Modell den älteren Teil zur Zusammenfassung zusammen. Das Original wandert nach `richterm-verlauf.archiv.md`; nichts geht verloren.

Beide Dateien darfst du bearbeiten: die Zusammenfassung kürzen, Wichtiges ergänzen, Falsches streichen. `history: false` im Profil schaltet das Gedächtnis ab.

7. Berechtigungen

Mit Claude Code kann die KI Werkzeuge benutzen: Dateien lesen und schreiben, Befehle ausführen, im Web suchen. Was ohne Nachfrage erlaubt ist, steuert der Berechtigungsmodus:

Modus	Verhalten	Wann sinnvoll
Fragen (<code>default</code>)	Lesen geht sofort; Schreiben und Befehle werden im Chat erfragt (Erlauben / Ablehnen).	Lernen, Unterlagen lesen, alles, wo nichts verändert werden soll.
Dateien bearbeiten erlauben (<code>acceptEdits</code>)	Dateien ändern ohne Nachfrage; Befehle werden weiter erfragt.	Programmieren im Projektordner.
Alles erlauben (<code>bypassPermissions</code>)	Keine Nachfragen.	Nur in Ordnern, in denen nichts kaputtgehen kann.

Feiner geht es mit `allowed_tools` / `disallowed_tools` im Profil, etwa `allowed_tools: Read Grep Glob` für einen Ordner, der nur gelesen werden darf.

8. Das Terminal und der Befehl `rt`

`Ctrl+Shift+T` öffnet ein Terminal im Arbeitsordner. Darin sind `rt`, `ollama` und `claude` direkt verfügbar. Mit `rt` schickst du Inhalte aus dem Terminal in den Chat:

```
rt ask 'Erkläre mir Backpropagation'      # Frage an den Chat
rt latex '\frac{1}{1+e^{-z}}'              # Formel als Karte im Chat
rt md notizen.md                          # Markdown-Datei (mit $...$-Formeln) anzeigen
rt image plot.png                        # Bild anzeigen (auch GIF)
rt html seite.html                       # HTML-Seite mit Animation einbetten
rt mermaid 'graph LR; A-->B'             # Diagramm
rt clear                                  # Chat-Karten entfernen
rt --help                                # alle Möglichkeiten
```

`rt` funktioniert auch in jedem anderen Terminal, solange RichTerm läuft. Startest du `claude` in einem RichTerm-Terminal, weiß es durch `~/.claude/CLAUDE.md`, dass es Formeln und Bilder mit `rt` in den Chat-Tab schicken kann.

9. Tastenkürzel

Taste	Wirkung
<code>Enter</code> / <code>Shift+Enter</code>	Senden / Zeilenumbruch (Chat)
<code>Ctrl</code> + <code>+</code> / <code>-</code>	Chat-Schrift größer / kleiner
<code>Ctrl+Shift+T</code>	Neues Terminal im Arbeitsordner
<code>Ctrl+Shift+W</code>	Terminal schließen
<code>Ctrl+PgUp</code> / <code>Ctrl+PgDn</code>	Tab wechseln
<code>Ctrl+Shift+Enter</code>	Zum Chat springen
<code>Ctrl+Shift+C</code> / <code>Ctrl+Shift+V</code>	Kopieren / Einfügen im Terminal (Rechtsklick fügt auch ein)
<code>Ctrl+Shift++</code> / <code>Ctrl+Shift+--</code>	Terminal-Schrift größer / kleiner
<code>Ctrl</code> + Klick auf Link	Link im Browser öffnen (Terminal)

10. Typische Arbeitsabläufe

Lernen mit Vorlesungsfolien

1. Ordner anlegen, z. B. `~/Uni/ML` mit Unterordnern `folien/`, `uebungen/`.
2. `cd ~/Uni/ML` und `richterm`.
3. *Profil* klicken: Rolle „Tutor“, Thema, Ordnerstruktur, „Quellen nennen“, Berechtigungen auf Fragen lassen. *Neu laden*.
4. Fragen: „Fass Folie 10–15 aus `vorlesung_03.pdf` zusammen und leite die Formel her.“ Claude liest das PDF und antwortet mit Formeln.
5. Am nächsten Tag wieder `richterm`: Die Sitzung wird fortgesetzt, der Verlauf steht im Chat.

Programmieren

1. `richterm ~/Projekte/mein-projekt`.
2. Profil: Rolle „Entwickler“, `permissions: acceptEdits`, ggf. Verbote (nichts löschen, keine Git-Pushes).
3. Aufgaben stellen; Werkzeugaufrufe erscheinen als graue Zeilen, Klick zeigt Details.

Kostenlos und privat mit lokalem Modell

1. Einmalig im RichTerm-Terminal: `ollama pull qwen2.5:7b`.
2. Im Profil `backend: command`, `model: qwen2.5:7b`, `context_chars: 6000`. *Neu laden*.
3. Fragen stellen; Antworten dauern länger und sind einfacher, aber nichts verlässt den Rechner.

11. Probleme und Lösungen

Problem	Lösung
Desktop-Icon tut nichts	Rechtsklick auf das Icon, „Starten erlauben“. Oder im Terminal <code>richterm</code> .
„Der Befehl claude wurde nicht gefunden“	Claude Code ist nicht installiert oder nicht im PATH. Im Terminal <code>claude --version</code> prüfen.
Lokales Modell antwortet nicht oder ewig	Im RichTerm-Terminal <code>ollama list</code> und <code>ollama ps</code> prüfen. Läuft ein alter System-Ollama auf 11434, stört er nicht; RichTerm benutzt Port 11435. Notfalls <code>kill -f "ollama serve"</code> für den eigenen Dienst; er startet beim nächsten Mal neu.
Antwort wird abgeschnitten / Prozess beendet	Statuszeile lesen; <i>Neuer Chat</i> starten. Der Verlauf bleibt in <code>richterm-verlauf.md</code> .
Profiländerung wirkt nicht	<i>Neu laden</i> klicken; gilt ab der nächsten Nachricht. Prüfen, ob das Feld oben (Einstellungen) oder unten (Text) gemeint ist.
Modell/Berechtigungen in der Kopfzeile gesperrt	Das Profil legt sie fest. Im Profil leer bzw. <code>default</code> eintragen, dann entscheidet die Kopfzeile.
Formeln werden nicht gerendert	Die KI muss LaTeX mit <code>\$\$...</code> schreiben; das steht im Profil. Bei lokalen Modellen hilft ein Hinweis in der Frage.
Kosten im Blick behalten	Die Statuszeile zeigt Dauer und Kosten jeder Antwort. Haiku ist am günstigsten.

12. Dateien im Überblick

Ort	Inhalt
<code>~/Desktop/RichTerm/</code>	Das Programm (Git-Repository): <code>richterm.py</code> , <code>chat/</code> , <code>bin/rt</code> , <code>vendor/</code> , <code>profile_template.md</code> , <code>start.sh</code> , <code>README.md</code> , <code>docs/</code> (diese Anleitung).
<code><Arbeitsordner>/richterm.md</code>	Profil für diesen Ordner.
<code><Arbeitsordner>/richterm-verlauf.md</code>	Gedächtnis; <code>richterm-verlauf.archiv.md</code> = verdichtete Rohteile.
<code>~/.config/richterm.json</code>	Fenstergröße, Schrift, letzter Ordner, gemerkte Claude-Sitzungen pro Ordner.
<code>~/.cache/richterm/html/</code>	Abgelegte HTML-Vorschauen (kann gelöscht werden).
<code>~/.local/share/ollama/</code>	Eigenes Ollama (<code>dist/</code>) und Modelle (<code>models/</code>).
<code>~/.local/bin/richterm</code> , <code>~/.local/bin/rt</code>	Die Befehle (Verweise ins Projekt).
<code>~/.claude/CLAUDE.md</code>	Globale Hinweise für Claude Code (u. a. zu <code>rt</code>).

